

【習題 4.3.1】

為何 Sarsa 是同策略，而 Q-learning 及 Expected Sarsa 是異策略？

答：

核心概念：

同策略（On-policy）：

- 評估與更新同一個行為策略。
- 學習時「執行的策略」＝「被學習的策略」。

異策略（Off-policy）：

- 學習的策略與執行的策略不同。
- 「執行策略」（行為策略）可探索，「學習策略」（目標策略）是最優的。

- Sarsa：學習的 $Q(s,a)$ 是根據 當前行為策略（behavior policy）選出的動作更新 → 同策略。

- Sarsa：「我照著現在這套策略走，學會它的結果。」
→ 所以是同策略。

- Q-learning：更新 Q 時用的是 最大 Q 值的動作，不一定是當前策略選的
→ 異策略。

- Q-learning：「我雖然現在隨機走，但我學習的是理論上最好的策略。」
→ 所以是異策略。

- Expected Sarsa：更新 Q 時用的是 行為策略下的期望值，更新方向可能與當前行為不同 → 異策略

- Expected Sarsa：「我根據目標策略的期望來更新，不看我實際怎麼走。」
→ 所以也是異策略。

結論: SARSA 是同策略，因為它學習並評估「實際執行的策略」。

Q-learning 與 Expected SARSA 是異策略，因為它們更新時用的是「不同於實際行為的目標策略」。

【習題 4.3.2】

(1).考慮[例題 1.4.2]有障礙物的迷宮以 Sarsa 演算法來求最佳策略

答:

π^* 最佳策略:

[G	←	←	←	↓	]
[↑	0	↑	0	↓	]
[↑	←	↑	↓	↓	]
[0	↑	0	→	↓	]
[→	→	→	→	G	]

(2). 考慮[例題 1.4.2]有障礙物的迷宮以 Q-learning 演算法來求最佳策略

答:

π^* 最佳策略:

[G	←	←	←	↓	]
[↑	0	↑	0	↓	]
[↑	←	→	↓	↓	]
[0	↓	0	↓	↓	]
[→	→	→	→	G	]

(3). 考慮[例題 1.4.2]有障礙物的迷宮以 Expected Sarsa 演算法來求最佳策略

答:

π^* 最佳策略:

[G	←	←	←	↓	]
[↑	0	↑	0	↓	]
[↑	←	↑	→	↓	]
[0	↑	0	→	↓	]
[→	→	→	→	G	]

【習題 4.3.3】

(1).考慮[例題 2.3.1] 有障礙及陷阱的迷宮，以 Sarsa 演算法來求最佳策略

答:

π^* 最佳策略:

[	↓		↓	→	↓	↓	]
[	↓		↓	T	↓	←	]
[	→		↓	↓	←	→	]
[	→	→	↓	0	0		]
[	T	→	→	→	G		]

(2).考慮[例題 2.3.1]有障礙及陷阱的迷宮，以 Q-learning 演算法來求最佳策略

答:

π^* 最佳策略:

[	→	↓		↓	↑	↓	↓	]
[	→	↓		↓	T	→	→	]
[	→	↓	→	↓	↓	←	↓	]
[	→	→	↓	↓	0	0		]
[	T	→	→	→	→	G		]

(3). 考慮[例題 2.3.1]有障礙及陷阱的迷宮，以 Expected Sarsa 演算法來求最佳策略

答:

π^* 最佳策略:

[	↓		↑	→	↑	←	]
[	↓		↓	T	→	↑	]
[	→		↓	↓	←	↑	]
[	→	→	↓	0	0		]
[	T	→	→	→	→	G	]

【習題 4.3.2】

(1).考慮[例題 1.4.2]有障礙物的迷宮以 Sarsa 演算法來求最佳策略

程式碼如下:

```
import numpy as np

nrow, ncol = 5, 5
discount, alpha = 0.95, 0.1      # 折扣因子  $\gamma$ , 學習率  $\alpha$ 
episode_num = 100000             # 總訓練回合數
init_eps, min_eps = 0.15, 0.01   #  $\epsilon$ -greedy 初始/最小  $\epsilon$ 
np.random.seed(42)
actions = np.array([0, 1, 2, 3])
action_word = np.array(["←", "↑", "→", "↓"])

moves = {0:(0,-1), 1:(-1,0), 2:(0,1), 3:(1,0)}
obstacle = [[1,1],[1,3],[3,0],[3,2]]
goal_states = [[0,0],[nrow-1,ncol-1]] # 終點 (兩個 goal)
Q = np.zeros((nrow, ncol, 4))

# 判斷是否 terminal (到達終點)
def is_terminal(s):
    return s in goal_states

# 判斷是否為有效狀態 (不超界、不在障礙物)
def valid_state(x, y):
    return 0 <= x < nrow and 0 <= y < ncol and [x,y] not
in obstacle

# 環境 step: 輸入 state + action, 輸出下個狀態與 reward
def step(state, a):
    x, y = state

    # 若在障礙物或 terminal, 不移動
    if state in obstacle or is_terminal(state):
        return state, -1 if state in obstacle else 0

    dx, dy = moves[a]
    nx, ny = x + dx, y + dy

    # 若越界或撞到障礙物 → 原地不動
    if not valid_state(nx, ny):
```

```

        nx, ny = x, y

    # reward: 到達終點 → 0; 其他 → -1
    reward = 0 if is_terminal([nx, ny]) else -1
    return [nx, ny], reward

# tie-break: 在 Q-value 相同時，選擇「離終點較近」的動作
def tie_break(state):
    x, y = state
    q_values = Q[x, y]

    # 取出 Q 值最大的行動們
    best = np.where(q_values == q_values.max())[0]

    # 終點座標 (右下角)
    gx, gy = nrow-1, ncol-1

    candidates = []
    for a in best:
        dx, dy = moves[a]
        nx, ny = np.clip(x+dx, 0, nrow-1), np.clip(y+dy,
0, ncol-1)

        # 若動作會撞障礙物，距離給一個很大的數
        if [nx,ny] in obstacle:
            candidates.append((a, 999))
            continue

        # 計算動作後的曼哈頓距離
        new_dist = abs(nx-gx) + abs(ny-gy)
        candidates.append((a, new_dist))

    # 選擇距離最小者
    candidates.sort(key=lambda t: t[1])
    return candidates[0][0]

# epsilon-greedy: 以  $\epsilon$  機率隨機走，以  $1-\epsilon$  選擇最佳動作
def eps_move(state, eps):

```

```

# 終點與障礙物狀態直接隨機選
if state in obstacle or is_terminal(state):
    return np.random.choice(actions)

# 隨機探索
if np.random.rand() < eps:
    return np.random.choice(actions)

# 否則選 tie-break (Q 值最大 + 距離最近)
return tie_break(state)

# SARSA 訓練
# 可用作起始點的狀態 (排除障礙物與終點)
valid_states = [[i,j] for i in range(nrow) for j in
range(ncol)
                 if [i,j] not in obstacle and not
is_terminal([i,j])]
for episode in range(episode_num):

    # epsilon 逐漸下降
    eps = max(min_eps, init_eps*(1 - episode/episode_num))
    # 從任意有效狀態開始
    state =
list(valid_states[np.random.randint(len(valid_states))])
    action = eps_move(state, eps)
    # 持續互動直到終點
    while not is_terminal(state):

        # 執行 action , 得到 next_state、reward
        next_state, reward = step(state, action)
        # SARSA: 下一個 action 使用同樣的策略選擇 (非 greedy Q-
learning)
        next_action = 0 if is_terminal(next_state) else
eps_move(next_state, eps)
        x, y = state
        nx, ny = next_state
        # SARSA update
        Q[x,y,action] += alpha * (

```

```

        reward + discount * Q[nx,ny,next_action] -
Q[x,y,action]
    )
    # 進入下一狀態
    state, action = next_state, next_action
arrow = np.full((nrow, ncol), '', dtype=object)
for i in range(nrow):
    for j in range(ncol):
        s = [i,j]
        if s in obstacle:
            arrow[i,j] = 'O'
        elif is_terminal(s):
            arrow[i,j] = 'G'
        else:
            arrow[i,j] = action_word[tie_break(s)] # 最佳動作箭頭
print("\n* 最佳策略:")
for row in arrow:
    print("[" + " ".join(f"{x:4}" for x in row), "]")

```

(2).考慮[例題 1.4.2]有障礙物的迷宮以 Q-learning 演算法來求最佳策略
程式碼如下:

```

import numpy as np

np.random.seed(543)
which = np.flatnonzero # 用於找出陣列中為 True 的索引
nrow, ncol = 5, 5
discount, alpha = 0.95, 0.1
tol_eps, prob_eps = 1e-5, 0.1
action_word = np.array(["left", "up", "right", "down"])
action_num = len(action_word)
actions = np.arange(action_num) # 動作索引陣列 [0,1,2,3]

# 障礙物座標
obstacle = [[1, 1], [1, 3], [3, 0], [3, 2]]

# 狀態-動作價值函數 Q(s,a)，初始化為全 0
Q = np.zeros((nrow, ncol, action_num))

```

```

# 判斷終點函數
is_terminal = lambda s: s in [[0, 0], [nrow-1, ncol-1]]

# 隨機起始狀態（非終點且非障礙物）
def get_initial():
    while True:
        s = [np.random.randint(nrow),
np.random.randint(ncol)] # 隨機生成一個座標
        if not is_terminal(s) and s not in obstacle:
            return s

# 環境轉換函數 step(s,a)
def step(state, a):
    if is_terminal(state):
        return state.copy(), 0 # 若已到終點，狀態不變，回報 0
    x, y = state
    # 定義動作對應的位移：左上右下
    dx, dy = [(0,-1), (-1,0), (0,1), (1,0)][a]
    # 計算下一狀態座標，並限制在邊界內
    nx, ny = np.clip([x+dx, y+dy], [0,0], [nrow-1,ncol-1])
    # 若下一狀態是障礙物，保持原地不動
    next_state = [nx, ny] if [nx, ny] not in obstacle else
state.copy()
    # 若到達終點回報 0，否則回報 -1
    reward = 0 if is_terminal(next_state) else -1
    return next_state, reward

def epsilon_greedy(state, Q, eps):
    i, j = state
    if np.random.rand() < eps:
        # 以機率  $\epsilon$  隨機選擇動作
        return int(np.random.choice(actions))
    # 否則選擇 Q 值最大的動作
    qvals = Q[i, j, :]
    # 找出最大的 Q 值索引，若有多個最大值則隨機選一個
    bests = which(np.abs(qvals - np.max(qvals)) < tol_eps)
    return int(np.random.choice(bests))

```



```

# Q-Learning 訓練
for _ in range(50000): # 訓練 50000 次 episode
    state = get_initial() # 隨機起始狀態
    while not is_terminal(state):
        a = epsilon_greedy(state, Q, probab_eps) # 選擇動作
        next_state, reward = step(state, a) # 執行動作，得到
        下一狀態與回報
        i, j = state
        ni, nj = next_state
        # Q-Learning 更新公式
        Q[i,j,a] += alpha * (reward + discount *
np.max(Q[ni,nj,:]) - Q[i,j,a])
        state = next_state # 移動到下一狀態

greedy_action = np.full((nrow, ncol), -1, dtype=int) # 初
始化策略矩陣
for i in range(nrow):
    for j in range(ncol):
        s = [i,j]
        if not is_terminal(s) and s not in obstacle:
            qvals = Q[i,j,:]
            bests = which(np.abs(qvals - np.max(qvals)) <
tol_eps)
            greedy_action[i,j] =
int(np.random.choice(bests)) # 選擇最佳動作

arrow_dict = {0:'←',1:'↑',2:'→',3:'↓'}
arrow_matrix = np.array([
    ['G' if is_terminal([i,j]) else 'O' if [i,j] in
obstacle else arrow_dict[greedy_action[i,j]]
    for j in range(ncol)]
    for i in range(nrow)
])
print("\n最佳策略:")
for row in arrow_matrix:
    print("[", " ".join(f"{x:2}" for x in row), "]")

```

(3).考慮[例題 1.4.2]有障礙物的迷宮以 Expected Sarsa 演算法來求最佳策略
程式碼如下:

```
import numpy as np
np.random.seed(42) # 設定隨機種子，確保每次執行結果一致

nrow, ncol = 5, 5
discount, alpha = 0.95, 0.1
episode_num = 100000 # 訓練總回合數
init_eps, min_eps = 0.15, 0.01 #  $\epsilon$ -greedy 初始與最小隨機探索率

actions = np.array([0, 1, 2, 3])
action_word = {0:"←", 1:"↑", 2:"→", 3:"↓"}

obstacle = {(1,1), (1,3), (3,0), (3,2)}
goals = {(0,0), (4,4)}

Q = np.zeros((nrow, ncol, 4))

def is_terminal(s):
    """檢查狀態 s 是否為終止狀態"""
    return tuple(s) in goals

def step(s, a):
    """
    根據當前狀態 s 和動作 a 執行一步
    回傳：下一狀態，即時獎勵 r
    """
    x, y = s
    # 如果在障礙物或終點，則不移動
    if (x,y) in obstacle or is_terminal(s):
        return s, -1 if (x,y) in obstacle else 0

    # 移動方向對應的增量 (左、上、右、下)
    move = [ (0,-1), (-1,0), (0,1), (1,0) ][a]
    nx, ny = np.clip([x+move[0], y+move[1]], [0,0], [nrow-1, ncol-1]) # 邊界限制
    # 若移動到障礙物，回到原位置
```

```

    if (nx,ny) in obstacle:
        nx, ny = x, y
    # 回傳下一狀態及獎勵 (走到目標 0, 其他步數 -1)
    return (nx,ny), (0 if (nx,ny) in goals else -1)

def tie_break_move(s, tol=0.2, pref=(1,2,3,0)):
    """
    Q 值相差很小時的平手策略，選擇最接近目標的動作
    pref: 動作優先順序，用於打破平手
    """
    x, y = s
    q = Q[x,y]
    best = np.where(q >= q.max() - tol)[0] # 找出接近最大 Q
    的動作
    candidates = []
    for a in best:
        dx, dy = [ (0,-1), (-1,0), (0,1), (1,0) ][a]
        nx, ny = np.clip([x+dx, y+dy], [0,0], [nrow-1,
ncol-1])
        if (nx,ny) in obstacle:
            candidates.append((a, 999, 99)) # 障礙物懲罰
            continue
        # 計算與最近目標的曼哈頓距離
        dist = min(abs(nx-gx)+abs(ny-gy) for gx,gy in
goals)
        prio = pref.index(a) # 動作優先順序
        candidates.append((a, dist, prio))
        # 根據距離與優先順序排序，返回最佳動作
    return sorted(candidates, key=lambda
t:(t[1],t[2]))[0][0]

# 可選起點的狀態
all_states = [(i,j) for i in range(nrow) for j in
range(ncol)
                if (i,j) not in obstacle and (i,j) not in
goals]
# Expected SARSA 訓練
for ep in range(episode_num):

```

```

#  $\epsilon$  隨訓練回合逐漸下降
eps = max(min_eps, init_eps * (1 - ep/episode_num))
s =
list(all_states[np.random.randint(len(all_states))]) # 隨機選起點

while not is_terminal(s):
    x, y = s
    #  $\epsilon$ -greedy 行為策略
    if np.random.rand() < eps:
        a = np.random.choice(actions) # 隨機探索
    else:
        a = tie_break_move(s) # 選擇當前最佳行動

    ns, r = step(s, a) # 執行動作
    nx, ny = ns
    qn = Q[nx, ny]

    # 目標策略 greedy，計算下一步期望 Q 值
    best = np.where(qn >= qn.max() - 1e-6)[0] # 多個最大值平均
    expected = sum((1/len(best) if i in best else 0) *
qn[i] for i in actions)

    # 更新 Q 值 (Expected SARSA 更新公式)
    Q[x, y, a] += alpha * (r + discount * expected -
Q[x, y, a])

    s = list(ns) # 移動到下一狀態
pi = np.full((nrow, ncol), "", dtype=object)
for i in range(nrow):
    for j in range(ncol):
        if (i,j) in obstacle:
            pi[i,j] = "O" # 障礙物
        elif (i,j) in goals:
            pi[i,j] = "G" # 目標
        else:
            pi[i,j] = action_word[tie_break_move((i,j))]

```

```
print("\nπ* 最佳策略：")
for row in pi:
    print("[" + " ".join(f"{x:4}" for x in row), ""]")
```

【習題 4.3.3】

(1).考慮[例題 2.3.1] 有障礙及陷阱的迷宮，以 Sarsa 演算法來求最佳策略

程式碼如下:

```
import numpy as np

np.random.seed(543)

# np.flatnonzero 可用來找出陣列中非零元素的索引，相當於篩選 True 的位置
which = np.flatnonzero
nrow, ncol = 5, 5
discount, alpha = 0.95, 0.01
eps, prob_eps = 1e-5, 0.1 # 用於浮點數比較的 epsilon 與 epsilon-greedy 的探索率
action = np.arange(4) # 動作索引：0~3 對應左上右下
action_word = np.array(["←", "↑", "→", "↓"])

goal = (4, 4) # 目標位置
obstacles = {(3, 3), (3, 4)} # 障礙物位置
traps = {(1, 2), (4, 0)} # 陷阱位置
start = (0, 0) # 起始位置

# 初始化 Q-table (列 x 行 x 動作數)
Q = np.zeros((nrow, ncol, 4))

# === 判斷是否為終止狀態 ===
def is_terminal(s):
    return tuple(s) == goal

# === 執行一步動作 ===
def step(s, a):
    x, y = s
    # 如果誤進障礙物，直接報錯
```

```

if (x, y) in obstacles:
    raise ValueError("Obstacle state reached!")

# 如果已經到達終點，回傳自身與獎勵 0
if is_terminal(s):
    return s, 0

# 如果踩到陷阱，回到起點並扣 50 分
if (x, y) in traps:
    return list(start), -50

# 動作對應的座標變化量
dx = [0, -1, 0, 1] # 左、上、右、下 x 方向變化
dy = [-1, 0, 1, 0] # 左、上、右、下 y 方向變化
nx, ny = x + dx[a], y + dy[a]

# 邊界限制
nx = np.clip(nx, 0, nrow - 1)
ny = np.clip(ny, 0, ncol - 1)

# 如果新位置是障礙物，留在原地
if (nx, ny) in obstacles:
    nx, ny = x, y

# 獎勵設定：達到終點 0 分，其他步數 -1
reward = 0 if is_terminal((nx, ny)) else -1
return [nx, ny], reward

# epsilon-greedy 策略選動作
def eps_greedy(s):
    x, y = s
    # 如果在障礙物、陷阱或終點，隨機選動作
    if (x, y) in obstacles or (x, y) in traps or
is_terminal(s):
        return np.random.choice(action)

# 以 prob_eps 的機率探索隨機動作
if np.random.rand() < prob_eps:

```

```

        return np.random.choice(action)

# 否則選擇 Q 值最大的動作，若有多個一樣最大，隨機選其中之一
q = Q[x, y]
best = which(np.abs(q - q.max()) < eps)
return np.random.choice(best)

# SARSA 訓練
for _ in range(10000):          # 訓練回合數
    s = list(start)            # 每回合從起點開始
    a = eps_greedy(s)          # 選擇初始動作

    while not is_terminal(s):   # 回合未結束
        ns, r = step(s, a)     # 執行動作並取得下一狀態與獎勵

        if is_terminal(ns):
            target = r          # 終點目標值
            na = None
        else:
            na = eps_greedy(ns) # 下一步動作
            target = r + discount * Q[ns[0], ns[1], na] #
SARSA 更新目標

        # Q-value 更新
        Q[s[0], s[1], a] += alpha * (target - Q[s[0],
s[1], a])
        # 移動到下一步
        s, a = ns, na if na is not None else (ns, None)[0]

greedy = np.argmax(Q, axis=2)          # 每個狀態選擇 Q 值最大
的動作
arrow = np.full((nrow, ncol), "", dtype=object)
for i in range(nrow):
    for j in range(ncol):
        if (i, j) in obstacles:
            arrow[i, j] = "O" # 障礙物
        elif (i, j) in traps:

```

```

        arrow[i, j] = "T" # 陷阱
    elif is_terminal((i, j)):
        arrow[i, j] = "G" # 目標
    else:
        arrow[i, j] = action_word[greedy[i, j]]
print("\n\n* 最佳策略:")
for row in arrow:
    print("[", " ".join(f"{x:6}" for x in row), "]")

```

(2).考慮[例題 2.3.1] 有障礙及陷阱的迷宮，以 Q-learning 演算法來求最佳策略
程式碼如下：

```

import numpy as np
np.random.seed(543) # 設定隨機種子，確保結果可重現
# 找出布林陣列為 True 的索引
which = lambda status: np.arange(len(status))[status]

nrow, ncol = 5, 5
discount, alpha = 0.95, 0.01
eps, prob_eps = 1e-5, 0.1 # 數值容差 eps、epsilon-greedy 隨機探索機率 prob_eps

action_word = np.array(["left", "up", "right", "down"])
action = np.arange(4) # 動作索引 [0,1,2,3]

goal = (4, 4) # 目標狀態
obstacles = {(3, 3), (3, 4)} # 障礙物
traps = {(1, 2), (4, 0)} # 陷阱
start = (0, 0) # 起始狀態

# ==== 初始化 Q(s,a) 與策略矩陣 ====
state_action_value = np.zeros((nrow, ncol,
len(action))) # Q(s,a) 初始化為 0
greedy_action = np.zeros_like(state_action_value) # 用於存放每個狀態的最佳動作

# ==== 判斷是否為終止狀態 ====

```



```

def is_terminal(state):
    return tuple(state) == goal # 到達目標即為終止

# ==== 執行動作函數 ====
def step(state, a):
    x, y = state
    # 如果在障礙物上，不應該發生
    if (x, y) in obstacles:
        raise ValueError(f"State {state} is an obstacle!")

    # 終止狀態不動作
    if is_terminal(state):
        return state, 0
    # 掉入陷阱，回到起點並給負獎勵
    if (x, y) in traps:
        return list(start), -50

    # 定義四個動作的位移
    moves = [(0, -1), (-1, 0), (0, 1), (1, 0)] # 左、上、
    # 右、下
    nx, ny = np.clip(x + moves[a][0], 0, nrow - 1),
    np.clip(y + moves[a][1], 0, ncol - 1)
    # 如果下一步撞到障礙物，則保持原地
    if (nx, ny) in obstacles:
        nx, ny = x, y

    # 一般移動的獎勵，若到達目標，reward = 0，否則 -1
    reward = 0 if is_terminal([nx, ny]) else -1
    return [nx, ny], reward
# ====  $\epsilon$ -greedy 選擇動作 ====
def eps_greedy_move(state, Q):
    x, y = state
    # 若終止狀態、陷阱或以 prob_eps 機率隨機選擇動作
    if is_terminal(state) or (x, y) in traps or
np.random.rand() < prob_eps:
        return int(np.random.choice(action))
    # 否則選擇 Q 值最大的動作（若有多個最大，隨機選擇）
    q_values = Q[x, y, :]

```

```

    max_idx = which(np.abs(q_values - np.max(q_values)) <
eps)
    return int(np.random.choice(max_idx))
# Q-learning 訓練
for episode in range(10000): # 訓練 10000 回合
    state = list(start) # 每回合從起點開始
    while not is_terminal(state):
        a = eps_greedy_move(state, state_action_value) #
選擇動作
        next_state, reward = step(state, a) # 執行動作
        # 計算下一狀態的最大 Q 值
        next_max = 0 if is_terminal(next_state) else
np.max(state_action_value[next_state[0],
next_state[1], :])
        # Q-learning 更新公式
        state_action_value[state[0], state[1], a] += alpha
* (reward + discount * next_max -
state_action_value[state[0], state[1], a])
        state = next_state

# 計算最佳策略
for i in range(nrow):
    for j in range(ncol):
        # 終止、障礙物、陷阱不更新策略
        if is_terminal([i, j]) or (i, j) in obstacles or
(i, j) in traps:
            continue
        # 找出 Q 值最大的動作索引
        max_idx = which(np.abs(state_action_value[i, j, :]
- np.max(state_action_value[i, j, :]))) < eps)
        greedy_action[i, j, max_idx] = 1 # 標記為最佳動作
arrow_dict = {0:'←', 1:'↑', 2:'→', 3:'↓'}
arrow_matrix = np.full((nrow, ncol), '', dtype=object)
for i in range(nrow):
    for j in range(ncol):
        if (i, j) in obstacles: arrow_matrix[i, j] =
'O' # 障礙物

```

```

        elif (i, j) in traps: arrow_matrix[i, j] =
'T'      # 陷阱
        elif is_terminal([i, j]): arrow_matrix[i, j] =
'G'      # 目標
        else: arrow_matrix[i, j] = ''.join([arrow_dict[k]
for k in which(greedy_action[i, j, :] == 1)]) # 最佳策略箭
頭
print("\n\n*最佳策略:")
for row in arrow_matrix:
    print("[", " ".join(f"{x:6}" for x in row), "]")

```

(3).考慮[例題 2.3.1] 有障礙及陷阱的迷宮，以 Expected Sarsa 演算法來求最佳策略

程式碼如下：

```

import numpy as np
np.random.seed(543) # 設定隨機種子，確保實驗可重現
which = np.flatnonzero # 找出陣列中為 True 的索引

nrow, ncol = 5, 5
discount, alpha = 0.95, 0.01 # 折扣因子  $\gamma$  與學習率  $\alpha$ 
prob_epsilon, epsilon = 0.1, 1e-5 #  $\epsilon$ -greedy 探索率與數值容差
action_num = 4
action_word = np.array(["left", "up", "right", "down"])
action = np.arange(action_num)

# 起點、終點、障礙物與陷阱座標
start = (0, 0)
goal = (4, 4)
obstacles = {(3, 3), (3, 4)}
traps = {(1, 2), (4, 0)}
# 初始化  $Q(s,a)$  值與策略
Q = np.zeros((nrow, ncol, action_num)) #
Q 表格
action_prob = np.full((nrow, ncol, action_num), 0.25) #
每個動作初始均等概率

```

```

greedy_action = np.zeros((nrow, ncol, action_num)) #
最佳策略矩陣

# ==== 狀態檢查函數 ====
def is_terminal(state):
    """判斷是否為終點狀態"""
    return tuple(state) == goal

# ==== 環境互動函數 ====
def step(state, a):
    """
    執行動作 a 後更新狀態並返回 reward
    state: [x, y]
    a: 動作索引
    """
    x, y = state

    # 如果在障礙物上，拋出錯誤
    if (x, y) in obstacles:
        raise ValueError(f"Obstacle at {state}")

    # 到達終點，無獎勵
    if is_terminal(state):
        return state, 0

    # 掉入陷阱，回到起點並懲罰
    if (x, y) in traps:
        return start, -50

    # 動作對應位移
    dx, dy = [(0, -1), (-1, 0), (0, 1), (1, 0)][a] # 左、上、右、
下
    nx, ny = np.clip(x+dx, 0, nrow-1), np.clip(y+dy, 0,
ncol-1) # 邊界限制

    # 如果下一步是障礙物，則不移動
    if (nx, ny) in obstacles:
        nx, ny = x, y

```

```

    # reward: 到達終點為 0，其他正常步驟為 -1
    reward = 0 if is_terminal((nx, ny)) else -1
    return [nx, ny], reward
# ====  $\epsilon$ -greedy 選動作函數 ====
def eps_greedy_move(state):
    """
    根據  $\epsilon$ -greedy 策略選擇動作
    """
    x, y = state

    # 終點或陷阱隨機動作
    if is_terminal(state) or (x, y) in traps:
        return np.random.choice(action)

    # 探索
    if np.random.rand() < probab_epsilon:
        return np.random.choice(action)
    # 利用: 選擇最大 Q 值的動作 (可能有多個)
    q_values = Q[x, y, :]
    max_idx = which(np.abs(q_values - q_values.max()) <
eps)
    return np.random.choice(max_idx)
# 訓練
episodes = 10000
for ep in range(episodes):
    state = list(start)
    while not is_terminal(state):
        # 選擇動作
        a = eps_greedy_move(state)
        # 執行動作
        next_state, r = step(state, a)

        # 計算下一狀態的期望 Q 值
        next_q = Q[next_state[0], next_state[1], :]
        next_exp = 0 if is_terminal(next_state) else
np.sum(action_prob[next_state[0], next_state[1], :] *
next_q)

```

```

        # Q 更新 (Expected SARSA)
        Q[state[0], state[1], a] += alpha * (r + discount
* next_exp - Q[state[0], state[1], a])

        # 更新策略
        q_values = Q[state[0], state[1], :]
        max_idx = which(np.abs(q_values - q_values.max())
< eps)

        n_best = len(max_idx)
        n_other = action_num - n_best

        if n_best == action_num: # 如果所有動作 Q 值相等，均等
分配
            action_prob[state[0], state[1], :] =
1/action_num
        else:
            action_prob[state[0], state[1], :] = prob_eps
/ n_other
            action_prob[state[0], state[1], max_idx] = (1-
prob_eps)/n_best
            state = next_state

        # 計算 V(s)
        V = np.max(Q, axis=2) # 狀態價值取 Q(s,a) 的最大值
        for i in range(nrow):
            for j in range(ncol):
                if is_terminal([i,j]) or (i,j) in obstacles or
(i,j) in traps:
                    continue
                max_idx = which(np.abs(Q[i,j,:] - Q[i,j,:].max())
< eps)
                greedy_action[i,j,:] = 0
                greedy_action[i,j,max_idx] = 1 # 將最大動作標記為 1

        arrow_dict = {0:'←',1:'↑',2:'→',3:'↓'}
        arrow_matrix = np.full((nrow,ncol), '', dtype=object)
        for i in range(nrow):
            for j in range(ncol):

```

```

        if (i,j) in obstacles:
            arrow_matrix[i,j] = 'O'
        elif (i,j) in traps:
            arrow_matrix[i,j] = 'T'
        elif is_terminal([i,j]):
            arrow_matrix[i,j] = 'G'
        else:
            arrow_matrix[i,j] = ''.join([arrow_dict[k] for
k in which(greedy_action[i,j,:]==1)])
print("\n\n*最佳策略:")
for row in arrow_matrix:
    print("[", " ".join(f"{x:6}" for x in row), "]")

```